#file:cpp

ADSA Experiments

ASSIGNMENT NO. 1

Problem Statement: Write C++ code for merge sort and analyze time complexity.

Aim: To understand and implement the Merge Sort algorithm.

Objective:

1. To implement the Merge Sort algorithm using the divide and conquer strategy.
2. To analyze its performance in terms of time and space complexity.

Theory / Background: What is Sorting? Sorting is the process of arranging data elements in a specific order.

Merge Sort Overview: Merge Sort is a divide and conquer algorithm that splits the input array into smaller subarrays until each contains a single element. It then recursively sorts the subarrays and merges them to produce the final sorted array.

Algorithm Steps:

1. Start
2. Find the middle index of the array to divide it into two halves.
3. Recursively apply Merge Sort to both halves until single-element subarrays are achieved.
4. Compare and merge elements of the sorted subarrays to form a single sorted array.
5. End

Code:

```
#include <iostream>
#include <vector>

using namespace std;

void merge(vector<int>& arr, int left, int mid, int r
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1);
    vector<int> R(n2);

    for (int i = 0; i < n1; i++) L[i] = arr[left + i]
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 +

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSort(vector<int>& arr, int left, int right)
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

void printArray(const vector<int>& arr) {
    for (int x : arr) cout << x << " ";
    cout << endl;
}

int main() {
    vector<int> arr = {12, 11, 13, 5, 6, 7};
    cout << "Original array: ";
    printArray(arr);
    mergeSort(arr, 0, arr.size() - 1);
    cout << "Sorted array:   ";
```

```
    printArray(arr);
    return 0;
}
```

Analysis:

- **Time Complexity:**
  - *Best Case:* O(n log n)
  - *Average Case:* O(n log n)
  - *Worst Case:* O(n log n)
- **Space Complexity:** O(n)
- **Stability:** Yes (Stable sort)

Conclusion: Merge Sort is a stable and efficient sorting algorithm with a consistent time complexity of `O(n log n)`.

ASSIGNMENT NO. 2

Problem Statement: Implement 0/1 Knapsack problem.

Aim: To understand and implement the 0/1 Knapsack problem using Dynamic Programming.

Objective:

1. To learn how to solve optimization problems using Dynamic Programming.
2. To maximize the total profit without exceeding the weight capacity of the knapsack.

Theory / Background: The 0/1 Knapsack problem is a combinatorial optimization problem. Given a set of items, each with a weight and a value, we must determine which items to include in a collection so that the total weight is less than or equal to a given limit (capacity) and the total value is as large as possible. In "0/1", you can either take an item entirely (1) or leave it (0).

Algorithm Steps:

1. Initialize a 2D array `dp` of size `(n+1) x (W+1)` with zeros, where `n` is the number of items and `W` is the capacity.
2. Loop through each item, and for each item, loop through all possible capacities from 1 to `W`.

3. If the item's weight is less than or equal to the current capacity, choose the maximum between including the item and not including it.
4. Otherwise, if the weight is strictly greater than the capacity, copy the value from the upper row.
5. `dp[n][W]` will hold the maximum possible value.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int knapSack(int W, const vector<int>& wt, const vect
    vector<vector<int>> dp(n + 1, vector<int>(W + 1,

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            } else if (wt[i - 1] <= w) {
                dp[i][w] = max(val[i - 1] + dp[i - 1]
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    return dp[n][W];
}

int main() {
    vector<int> val = {60, 100, 120};
    vector<int> wt = {10, 20, 30};
    int W = 50;
    int n = val.size();

    cout << "Maximum value that can be put in a knaps
    return 0;
}
```

Analysis:

- **Time Complexity:** O(n * W)
- **Space Complexity:** O(n * W)

Conclusion: We successfully implemented the 0/1 Knapsack Problem using Dynamic Programming to find the optimal maximum value within a bounded weight capacity.

ASSIGNMENT NO. 3

Problem Statement: Implement Cuckoo Hashing.

Aim: To understand and implement Cuckoo Hashing to resolve collisions.

Objective:

1. To understand the mechanism of hashing with multiple hash functions.
2. To resolve collisions aggressively by displacing previous keys.

Theory / Background: Cuckoo hashing is a scheme for resolving hash collisions using two hash functions and two hash tables. When inserting a new key, if the primary slot in the first table is occupied, the old key is pushed out (like a cuckoo bird pushes eggs out of a nest) and moved to its alternative slot in the second table. This displacement repeats until a vacant spot is found or a cycle is detected.

Algorithm Steps:

1. Use two hash functions `hash1` and `hash2` and two arrays `table1` and `table2`.
2. To insert a key, compute its position using `hash1`.
3. If the position is empty, place the key.
4. If occupied, push the old key out, insert the new one, and then try to insert the pushed-out key into `table2` using `hash2`.
5. Repeat this bouncing process until everything settles or a maximum number of bounces is reached (rehashing would be required in this case).

Code:

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

class CuckooHash {
private:
    int size;
    vector<int> table1;
    vector<int> table2;

    int hash1(int key) { return key % size; }
    int hash2(int key) { return (key / size) % size;

public:
    CuckooHash(int tableSize) : size(tableSize) {
        table1.assign(size, -1);
        table2.assign(size, -1);
    }

    void insert(int key) {
        int targetKey = key;
        for (int i = 0; i < size; ++i) {
            int pos1 = hash1(targetKey);
            if (table1[pos1] == -1) {
                table1[pos1] = targetKey;
                return;
            }
            // Swap and push out
            swap(targetKey, table1[pos1]);

            int pos2 = hash2(targetKey);
            if (table2[pos2] == -1) {
                table2[pos2] = targetKey;
                return;
            }
            // Swap and push out
            swap(targetKey, table2[pos2]);
        }
        cout << "Cycle detected for key " << key << "
    }

    void display() {
        cout << "Table 1: ";
        for (int x : table1) cout << (x == -1 ? "-" :
        cout << "\nTable 2: ";
        for (int x : table2) cout << (x == -1 ? "-" :
        cout << endl;
    }
```

```
};

int main() {
    CuckooHash ch(5); // Table sizes of 5
    int keys[] = {20, 50, 53, 75, 100};

    for (int key : keys) {
        ch.insert(key);
    }

    ch.display();
    return 0;
}
```

Analysis:

- **Time Complexity:**
  - *Best Case:* O(1) for insertion and lookup.
  - *Worst Case:* O(n) for insertion if rehashing happens.
- **Space Complexity:** O(n)

Conclusion: Cuckoo hashing provides guaranteed O(1) time lookups and deletions by relocating entries upon collision.

ASSIGNMENT NO. 4

Problem Statement: Implement any real time application with the help of hashing concept and handle collision using linear probing without replacement.

Aim: To implement a real-time Hash Table application handling collisions via Linear Probing without Replacement.

Objective:

1. To understand collision handling using Linear Probing.
2. To create a practical hashing application, such as a localized Student Database mapping Roll Number to Names.

Theory / Background: Hashing is used to retrieve data in O(1) average time. Linear Probing is an open-addressing strategy where if a collision occurs at index `i`, we linearly probe `i+1`, `i+2`, ... to find the next available empty spot. 'Without replacement' means we never move an element once it is placed in the table.

Algorithm Steps:

1. Define a Hash Table size.
2. For an input key, calculate index = `key % size`.
3. If `table[index]` is unoccupied, place the key.
4. If occupied, incrementally check `(index + i) % size` until an empty slot is found.
5. Insert the element in that available spot.

Code:

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

class StudentRecord {
public:
    int roll;
    string name;
    StudentRecord() : roll(-1), name("") {}
    StudentRecord(int r, string n) : roll(r), name(n)
};

class HashTable {
private:
    int size;
    vector<StudentRecord> table;

public:
    HashTable(int s) : size(s) {
        table.assign(size, StudentRecord());
    }

    int hashFunc(int key) {
        return key % size;
    }

    void insert(int roll, string name) {
        int index = hashFunc(roll);
        int i = 0;

        while (table[(index + i) % size].roll != -1)
            if (table[(index + i) % size].roll == rol
                cout << "Roll number already exists."
                return;
            }
            i++;
            if (i == size) {
                cout << "Hash Table is Full!" << endl
                return;
            }
        }

        table[(index + i) % size] = StudentRecord(rol
        cout << "Inserted " << name << " at index " <
    }

    void display() {
        for (int i = 0; i < size; i++) {
```

```
            if (table[i].roll != -1)
                cout << "Index " << i << ": Roll = "
            else
                cout << "Index " << i << ": Empty" <<
        }
    }
};

int main() {
    HashTable ht(5);
    ht.insert(10, "Arnav");
    ht.insert(15, "Bob");      // Collision with 10 if
    ht.insert(20, "Alice");    // Collision again
    ht.display();
    return 0;
}
```

Analysis:

- **Time Complexity:**
  - *Best Case:* O(1)
  - *Average/Worst Case:* O(n) (when clustering occurs).
- **Space Complexity:** O(n)

Conclusion: Linear probing successfully resolves collisions, but may lead to primary clustering when the load factor gets high.

ASSIGNMENT NO. 5

Problem Statement: A Dictionary stores keywords & its meanings. Provide facility for adding new keywords, deleting keywords, updating values of any entry. Provide facility to display whole data sorted in ascending/ Descending order. Also find how many maximum comparisons may require for finding any keyword. Use Binary Search Tree for implementation.

Aim: To implement a dynamic Dictionary using a Binary Search Tree (BST).

Objective:

1. To use a BST to store Key-Value pairs efficiently.
2. To allow fast insertion, retrieval, and ordered traversal (Ascending / Descending).

Theory / Background: A Binary Search Tree uniquely maintains an ordering property: the left subtree contains smaller keys, and the right subtree contains larger keys. Therefore, an Inorder traversal results in sorting in Ascending Order, while a reverse Inorder traversal results in Descending Order. The maximum comparisons required for a lookup is mathematically equivalent to the height of the BST.

Algorithm Steps:

1. Create a recursive Node structure holding a Keyword, Meaning, Left Child, and Right Child.
2. **Insertion:** Navigate left if smaller, right if larger, until finding a null spot to insert.
3. **Ascending:** Process Left -> Node -> Right.
4. **Descending:** Process Right -> Node -> Left.
5. **Max Comparisons:** Track the depth/height of the tree recursively.

Code:

```
#include <iostream>
#include <string>
#include <algorithm>

using namespace std;

struct Node {
    string keyword, meaning;
    Node *left, *right;
    Node(string k, string m) : keyword(k), meaning(m)
};

class Dictionary {
    Node* root;

    Node* insertRec(Node* root, string k, string m) {
        if (!root) return new Node(k, m);
        if (k < root->keyword) root->left = insertRec
        else if (k > root->keyword) root->right = ins
        else root->meaning = m; // Update existing
        return root;
    }

    void inorderAsc(Node* root) {
        if (root) {
            inorderAsc(root->left);
            cout << root->keyword << " : " << root->m
            inorderAsc(root->right);
        }
    }

    void inorderDesc(Node* root) {
        if (root) {
            inorderDesc(root->right);
            cout << root->keyword << " : " << root->m
            inorderDesc(root->left);
        }
    }

    int getHeight(Node* root) {
        if (!root) return 0;
        return 1 + max(getHeight(root->left), getHeig
    }

public:
    Dictionary() : root(nullptr) {}
    void add(string k, string m) { root = insertRec(r
    void displayAscending() { cout << "Ascending:\n";
    void displayDescending() { cout << "Descending:\n
    int maxComparisons() { return getHeight(root); }
```

```
};

int main() {
    Dictionary dict;
    dict.add("Apple", "A fruit");
    dict.add("Zebra", "An animal");
    dict.add("Mango", "King of fruits");
    dict.add("Apple", "A red fruit"); // Update test

    dict.displayAscending();
    cout << "\n";
    dict.displayDescending();

    cout << "\nMax comparisons needed for any keyword
    return 0;
}
```

Analysis:

- **Time Complexity:**
  - *Average Case (Insertion/Search):* O(log n)
  - *Worst Case (Skewed Tree):* O(n)
- **Space Complexity:** O(n) for the tree nodes plus O(h) for recursion depth.

Conclusion: Using a BST provides a scalable and automatically ordered structure that effectively acts as a dynamic dictionary.

ASSIGNMENT NO. 6

Problem Statement: Construct inorder TBT (Threaded Binary Tree) and apply nonrecursive traversals.

Aim: To construct an Inorder Threaded Binary Tree and traverse it without using recursion or a stack.

Objective:

1. To understand exactly how Threaded Binary Trees optimize space and traversal time.
2. To link the null right pointers to their inorder successors.

Theory / Background: Standard BST traversals demand a stack or recursion to remember unvisited ancestors. A Threaded Binary Tree (TBT) repurposes empty `right` pointers to point directly to the

node's Inorder Successor, allowing for purely iterative O(1) space traversal. A boolean flag specifies whether the right pointer is a normal child or a 'thread'.

Algorithm Steps:

1. Node structure must hold a `isThreaded` boolean flag.
2. Determine where to insert the new Node (Standard BST logic).
3. **Update pointers:** If inserted as a left child, its thread becomes the parent. If inserted as a right child, its thread inherits the parent's thread.
4. **Traversal:** Start at the leftmost node. If it has a thread, jump right using the thread. If it has a right child, jump right and then go leftmost again.

Code:

```
#include <iostream>

using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    bool isThreaded;

    Node(int val) : data(val), left(nullptr), right(n
};

class ThreadedBST {
private:
    Node* root;

    Node* leftmost(Node* node) {
        if (!node) return nullptr;
        while (node->left) node = node->left;
        return node;
    }

public:
    ThreadedBST() : root(nullptr) {}

    void insert(int key) {
        Node* newNode = new Node(key);
        if (!root) {
            root = newNode;
            return;
        }

        Node* curr = root;
        Node* parent = nullptr;
        while (curr) {
            parent = curr;
            if (key < curr->data) {
                curr = curr->left;
            } else if (!curr->isThreaded) {
                curr = curr->right;
            } else {
                break;
            }
        }

        if (key < parent->data) {
            parent->left = newNode;
            newNode->right = parent;
            newNode->isThreaded = true;
```

```
        } else {
            newNode->right = parent->right;
            newNode->isThreaded = parent->isThreaded;
            parent->isThreaded = false;
            parent->right = newNode;
        }
    }

    void inorder() {
        Node* curr = leftmost(root);
        while (curr) {
            cout << curr->data << " ";
            if (curr->isThreaded) {
                curr = curr->right;
            } else {
                curr = leftmost(curr->right);
            }
        }
        cout << endl;
    }
};

int main() {
    ThreadedBST tbt;
    tbt.insert(20);
    tbt.insert(10);
    tbt.insert(30);
    tbt.insert(5);
    tbt.insert(15);

    cout << "Inorder Traversal of TBT: ";
    tbt.inorder();

    return 0;
}
```

Analysis:

- **Time Complexity:**
  - *Insertion:* O(log n) average.
  - *Traversal:* O(n)
- **Space Complexity:** O(1) auxiliary space (no recursion or stack needed).

Conclusion: TBT successfully eliminates the need for stacks or recursion during inorder traversal by leveraging unused null pointers.

ASSIGNMENT NO. 7

Problem Statement: From given postfix expression construct expression tree and traverse it using inorder traversal (non-recursive).

Aim: To construct an Expression Tree from a Postfix notation and perform an iterative Inorder traversal.

Objective:

1. To parse postfix strings into tree nodes sequentially.
2. To utilize a stack of tree nodes to build the expression tree.
3. To traverse this tree iteratively to retrieve the infix form.

Theory / Background: An Expression Tree maps algebraic expressions where leaves are operands and internal nodes are operators. Since postfix expressions already specify execution order perfectly, they inherently imply the structure of the tree. By using an explicit Stack of Pointers during traversal, we can simulate recursion entirely.

Algorithm Steps:

1. Scan the postfix expression from left to right.
2. If character is an operand, push a new Node directly to stack.
3. If an operator, pop two Nodes from the stack.
4. Set the operator Node's right child to the first popped node, and left child to the second.
5. Push the operator Node back onto the stack.
6. **Non-recursive Inorder traversal:** Iterate left deeply, pushing to stack. Pop and print, jump right.

Code:

```
#include <iostream>
#include <stack>
#include <string>
#include <cctype>

using namespace std;

struct Node {
    char data;
    Node* left;
    Node* right;
    Node(char val) : data(val), left(nullptr), right(
};

class ExpressionTree {
public:
    Node* buildTree(string postfix) {
        stack<Node*> st;
        for (char ch : postfix) {
            if (isalnum(ch)) {
                st.push(new Node(ch));
            } else {
                Node* root = new Node(ch);
                root->right = st.top(); st.pop();
                root->left = st.top(); st.pop();
                st.push(root);
            }
        }
        return st.top();
    }

    void inorderIterative(Node* root) {
        stack<Node*> st;
        Node* curr = root;

        while (curr != nullptr || !st.empty()) {
            while (curr != nullptr) {
                st.push(curr);
                curr = curr->left;
            }
            curr = st.top();
            st.pop();
            cout << curr->data;
            curr = curr->right;
        }
        cout << endl;
    }
};

int main() {
```

```
    ExpressionTree et;
    string postfix = "ab+c*d/";
    cout << "Postfix Expression: " << postfix << endl

    Node* root = et.buildTree(postfix);

    cout << "Inorder (Infix) Traversal: ";
    et.inorderIterative(root);

    return 0;
}
```

Analysis:

- **Time Complexity:**
  - *Tree Construction:* O(n)
  - *Non-Recursive Traversal:* O(n)
- **Space Complexity:** O(n) auxiliary space due to the stack.

Conclusion: A postfix string naturally lends itself safely to building an expression tree, and using a stack effectively allows non-recursive execution of classic tree traversals.